

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Решение систем линейных алгебраических уравнений методом Гаусса–Жордана»

Выполнил:

студент группы 3825Б1ПМиИ1
Булганов Даниил Александрович

Проверила:

Бусько П.В.

Содержание

Введение	3
Постановка задачи	3
Руководство пользователя	4
Описание программной реализации.....	5
Результаты экспериментов	8
Заключение	12
Литература	13
Приложение	144

Введение

Тема решения систем линейных алгебраических уравнений — одна из ключевых в вычислительной математике и программировании. Её решение заключается в нахождении неизвестных переменных, удовлетворяющих заданным линейным соотношениям. Основным критерий, по которому различаются методы решения, — это их точность, устойчивость и скорость работы.

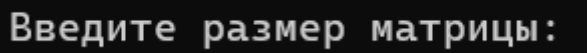
Практическое и теоретическое значение этой задачи трудно переоценить. С учебной точки зрения, реализация метода Гаусса–Жордана позволяет закрепить навыки работы с динамическими структурами данных, шаблонами классов и наследованием в C++. С практической же стороны, решение СЛАУ лежит в основе огромного количества прикладных систем: от расчёта конструкций до машинного обучения. От того, насколько устойчиво и правильно реализован алгоритм, зависит достоверность получаемых результатов.

Постановка задачи

В рамках данной лабораторной работы требуется реализовать на языке C++ метод Гаусса для решения систем линейных алгебраических уравнений с вещественной квадратной матрицей, использующий выбор ведущего элемента для повышения численной устойчивости. Необходимо решить систему линейных уравнений вида $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, где $\mathbf{A}$ — квадратная матрица коэффициентов, $\mathbf{b}$ — вектор правой части, $\mathbf{x}$ — искомый вектор неизвестных. Для этого требуется разработать шаблонный класс `Vector`, обеспечивающий динамическое управление памятью и доступ к элементам. На его основе реализовать класс квадратной матрицы, представляющий собой шаблон класса «вектор от вектора». Затем создать класс СЛАУ, наследующий квадратную матрицу, который содержит метод Гаусса, принимающий правую часть в качестве аргумента и возвращающий решение в виде вектора. Для удобства работы допускается реализовать вспомогательную функцию `swap` для обмена строк и элементов. Разработанное консольное приложение должно обеспечивать корректное решение СЛАУ с выводом результатов на экран.

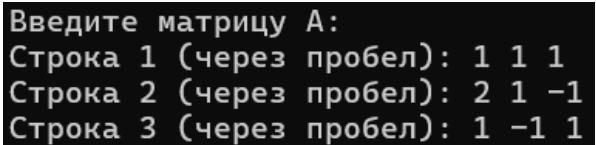
Руководство пользователя

При запуске программы перед пользователем возможность ввести размер квадратной матрицы.(рис.1).



Введите размер матрицы: (рис.1)

Пользователь вводит целое положительное число. Затем программа просит ввести матрицу A построчно.(рис.2)



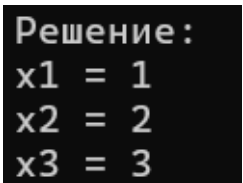
Введите матрицу A:
Строка 1 (через пробел): 1 1 1
Строка 2 (через пробел): 2 1 -1
Строка 3 (через пробел): 1 -1 1 (рис.2)

Далее вводится вектор правой части b. (рис.3).



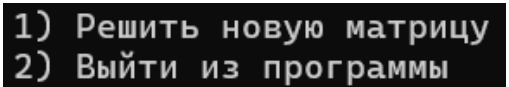
Введите вектор правой части b (через пробел):6 1 2 (рис.3)

Программа выводит решение системы.(рис.4)



Решение:
x1 = 1
x2 = 2
x3 = 3 (рис.4)

После решения предлагается меню.(рис.5)



1) Решить новую матрицу
2) Выйти из программы (рис.5)

При вводе цифры 1 программа предложит ввести новую матрицу для ее решения. При вводе цифры 2 программа завершит свою работу.

Описание программной реализации

В программе содержится 2 файла: Lab_Work_3.cpp, slau.h

В файле slau.h подключены библиотеки math.h и iostream, а также расписаны все функции, которые используются в программе. Ниже описал все функции подробно.

Класс **Vector** реализует динамический массив (вектор) элементов произвольного типа **T**. Он обеспечивает автоматическое управление памятью и базовые операции для работы с последовательностью данных.

Конструктор **Vector(int n = 0)** — принимает размер вектора и выделяет динамическую память под **n** элементов. Каждый элемент инициализируется значением по умолчанию **T()**. Если размер не указан, создаётся пустой вектор.

Деструктор **~Vector()** — освобождает выделенную динамическую память, предотвращая утечки памяти.

Конструктор копирования **Vector(const Vector& other)** — создаёт новый вектор как точную копию существующего. Выделяет новую память и копирует все элементы из исходного вектора.

Оператор присваивания **operator=** — выполняет глубокое копирование содержимого одного вектора в другой. Проверяет самоприсваивание, освобождает старую память, выделяет новую и копирует элементы.

Метод **size()** — возвращает текущее количество элементов в векторе. Ничего не изменяет.

Оператор **operator[]** (две версии) — предоставляет доступ к элементам вектора по индексу. Первая версия возвращает ссылку на элемент (позволяет изменять значение), вторая версия предназначена для константных объектов и не позволяет изменять элементы.

Метод **swap(Vector& other)** — реализует быстрый обмен содержимым между двумя векторами. Механизм основан на обмене указателями на динамические массивы и значениями длины. Это позволяет обменять данные за $O(1)$ без поэлементного копирования.

Класс **Matrix** реализует квадратную матрицу произвольного типа **T**, построенную как «вектор из векторов». Содержит защищённые поля: **data** (объект типа **Vector<Vector<T>>**) и **size_n** (размер матрицы).

Конструктор **Matrix(int n = 0)** — принимает размер матрицы и создаёт квадратную матрицу размером $n \times n$. Если передан отрицательный размер, он заменяется на 0. Выделяет память под **n** строк, каждая из которых является вектором длины **n**.

Метод **razmer()** — возвращает размер матрицы. Ничего не изменяет.

Оператор **operator[]** (две версии) — предоставляет доступ к строке матрицы по индексу, возвращая объект **Vector<T>**. Первая версия позволяет изменять строки, вторая предназначена для константных объектов.

Метод **swapRows(int row1, int row2)** — выполняет обмен двух строк матрицы местами. Принимает индексы строк для обмена. Проверяет корректность индексов и выполняет обмен с помощью метода **swap** класса **Vector**. Если строки совпадают или индексы некорректны, ничего не происходит.

Класс `SLAU` предназначен для решения систем линейных алгебраических уравнений. Он наследует класс `Matrix<T>` и добавляет метод решения методом Гаусса–Жордана.

Конструктор `SLAU(int n)` — вызывает конструктор базового класса `Matrix<T>` и создаёт квадратную матрицу размером $n \times n$.

Метод `gauss_jordan(Vector<T> pravo)` — реализует метод Гаусса–Жордана с выбором главного элемента для решения системы $Ax = b$. Принимает в качестве аргумента вектор правой части `pravo` и возвращает вектор неизвестных x .

В файле `Lab_Work_3.cpp` есть только одна главная функция `main`. Она служит для организации взаимодействия с пользователем. Функция `setlocale(LC_ALL, "RU")` дает возможность корректно выводить сообщение на русском языке, после чего входит в цикл `do...while`, позволяющий решать несколько систем подряд. Внутри цикла программа запрашивает размер матрицы, проверяя его положительность, затем создаёт объект `SLAU` указанного размера и заполняет матрицу коэффициентов и вектор правой части, вводимые пользователем. Далее вызывается метод `gauss_jordan`, решение выводится на экран, после чего пользователю предлагается выбрать: решить новую матрицу или выйти из программы. При вводе некорректного значения программа повторяет запрос. При выборе выхода цикл завершается и программа заканчивает работу.

Результаты экспериментов

Для проверки корректности работы реализованного метода Гаусса–Жордана было проведено тестирование на нескольких системах линейных алгебраических уравнений. Все расчёты выполнены с использованием типа `double`. Во всех примерах программно вычисленное решение сравнивалось с аналитическим или полученным методом подстановки.

Рассматривается система следующего вида:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8 \\ -3x_1 - x_2 + 2x_3 = -11 \\ -2x_1 + x_2 + 2x_3 = -3 \end{cases}$$

В матричной форме $A \cdot x = b$:

$$A = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}$$

Решение, полученное программой:

```
Введите размер матрицы: 3
Введите матрицу A:
Строка 1 (через пробел): 2 1 -1
Строка 2 (через пробел): -3 -1 2
Строка 3 (через пробел): -2 1 2
Введите вектор правой части b (через пробел): 8 -11 -3

Решение:
x1 = 2
x2 = 3
x3 = -1
```

Подстановкой легко убедиться, что найденное решение удовлетворяет всем уравнениям системы.

Система размерностью 5*5:

$$\begin{cases} 2x_1 - x_2 + 3x_3 + 2x_5 = 22 \\ -2x_1 + 2x_2 + 3x_3 - x_5 = -1 \\ 3x_1 + 3x_2 - 2x_3 + x_4 = -2 \\ -3x_1 + 3x_2 + x_3 - 3x_4 = -9 \\ -4x_1 + x_2 + 4x_3 - 4x_4 - 2x_5 = -9 \end{cases}$$

Матрица коэффициентов A и вектор правой части b:

$$A = \begin{pmatrix} 2 & -1 & 3 & 0 & 2 \\ -2 & 2 & 3 & 0 & -1 \\ 3 & 3 & -2 & 1 & 0 \\ -3 & 3 & 1 & -3 & 0 \\ -4 & 1 & 4 & -4 & -2 \end{pmatrix} \quad b = \begin{pmatrix} 22 \\ -1 \\ -2 \\ -9 \\ -9 \end{pmatrix}$$

Решение, полученное программой:

```
Введите размер матрицы: 5
Введите матрицу A:
Строка 1 (через пробел): 2 -1 3 0 2
Строка 2 (через пробел): -2 2 3 0 -1
Строка 3 (через пробел): 3 3 -2 1 0
Строка 4 (через пробел): -3 3 1 -3 0
Строка 5 (через пробел): -4 1 4 -4 -2
Введите вектор правой части b (через пробел): 22 -1 -2 -9 -9

Решение:
x1 = 2
x2 = -1
x3 = 3
x4 = 1
x5 = 4
```

Подстановкой можно убедиться, что $x_1=2$, $x_2=-1$, $x_3=3$, $x_4=1$, $x_5=4$ удовлетворяет всем уравнениям системы.

Вырожденная матрица:

$$\begin{cases} x_1 + x_2 + x_3 = 6 \\ 2x_1 + 2x_2 + 2x_3 = 12 \\ x_1 + 2x_2 + 3x_3 = 14 \end{cases} \quad A = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 2 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 6 \\ 12 \\ 14 \end{pmatrix}$$

Вторая строка матрицы равна первой строке, умноженной на 2, поэтому определитель матрицы равен нулю и система не имеет единственного решения. При попытке решить такую систему программа выдала сообщение об ошибке:

```
Введите размер матрицы: 3
Введите матрицу A:
Строка 1 (через пробел): 1 1 1
Строка 2 (через пробел): 2 2 2
Строка 3 (через пробел): 1 2 3
Введите вектор правой части b (через пробел): 6 12 14
Ошибка: матрица вырождена или система не имеет единственного решения
```

Нулевая строка в матрице при ненулевом элементе правой части:

$$\begin{cases} x_1 + x_2 + x_3 = 3 \\ 0x_1 + 0x_2 + 0x_3 = 5 \\ x_1 - x_2 + 2x_3 = 4 \end{cases}$$

В матричной форме:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 5 \\ 4 \end{pmatrix}$$

Второе уравнение имеет вид $0=5$, что является противоречием. Такая система не имеет решений. При попытке решить её программа в процессе выполнения метода Гаусса–Жордана обнаруживает, что ведущий элемент не может быть выбран (все элементы в столбце ниже и включая текущий равны нулю, а соответствующий элемент правой части ненулевой), и выдаёт сообщение об ошибке:

```
Введите размер матрицы: 3
Введите матрицу A:
Строка 1 (через пробел): 1 1 1
Строка 2 (через пробел): 0 0 0
Строка 3 (через пробел): 1 -1 2
Введите вектор правой части b (через пробел): 3 5 4
Ошибка: матрица вырождена или система не имеет единственного решения
```

Система с нецелыми коэффициентами:

$$\begin{cases} 2,5x_1 + 1,2x_2 - 0,5x_3 = 7,1 \\ -1,5x_1 + 3,4x_2 + 1,2x_3 = 5,8 \\ 0,8x_1 - 0,3x_2 + 4,1x_3 = 12,3 \end{cases}$$

Матрица коэффициентов A и вектор правой части b:

$$A = \begin{pmatrix} 2,5 & 1,2 & -0,5 \\ -1,5 & 3,4 & 1,2 \\ 0,8 & -0,3 & 4,1 \end{pmatrix}, \quad b = \begin{pmatrix} 7,1 \\ 5,8 \\ 12,3 \end{pmatrix}$$

Решение, полученное программой:

```
Введите размер матрицы: 3
Введите матрицу A:
Строка 1 (через пробел): 2.5 1.2 -0.5
Строка 2 (через пробел): -1.5 3.4 1.2
Строка 3 (через пробел): 0.8 -0.3 4.1
Введите вектор правой части b (через пробел): 7.1 5.8 12.3

Решение:
x1 = 2.47674
x2 = 1.86221
x3 = 2.65299
```

Этот пример демонстрирует, что программа корректно работает не только с целыми, но и с дробными числами.

Заключение

В ходе выполнения данной лабораторной работы был реализован метод Гаусса–Жордана с выбором ведущего элемента для решения систем линейных алгебраических уравнений. Для этого были разработаны шаблонный класс `Vector`, обеспечивающий динамическое управление памятью и доступ к элементам, шаблонный класс `Matrix`, представляющий собой квадратную матрицу, построенную как «вектор из векторов», а также класс `SLAU`, наследующий `Matrix` и содержащий метод `gauss_jordan`, который принимает вектор правой части и возвращает вектор решения.

Проведённое тестирование на системах размерности 3×3 и 5×5 подтвердило корректность работы алгоритма — полученные решения совпали с ожидаемыми с высокой точностью. Также программа успешно справилась с решением системы, содержащей нецелые коэффициенты, что подтверждает её работоспособность для вещественных чисел. Кроме того, программа корректно обрабатывает вырожденные и несовместные системы, выдавая соответствующие сообщения об ошибке.

Таким образом, все поставленные задачи были выполнены: реализованы требуемые классы, метод Гаусса–Жордана работает корректно, интерфейс программы удобен для пользователя и позволяет решать несколько систем подряд без перезапуска.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение

```
int main() {
    setlocale(LC_ALL, "RU");

    int choice;

    do {
        int n;
        cout << "Введите размер матрицы: ";
        cin >> n;

        if (n <= 0) {
            cout << "Ошибка: размер матрицы должен быть положительным\n";
            continue;
        }

        SLAU<double> slau(n);

        cout << "Введите матрицу A:\n";
        for (int i = 0; i < n; i++) {
            cout << "Строка " << i + 1 << " (через пробел): ";
            for (int j = 0; j < n; j++) {
                cin >> slau[i][j];
            }
        }

        Vector<double> b(n);
        cout << "Введите вектор правой части b (через пробел):";
        for (int i = 0; i < n; i++) {
            cin >> b[i];
        }

        Vector<double> x = slau.gauss_jordan(b);

        cout << "\nРешение:\n";
        for (int i = 0; i < n; i++) {
            cout << "x" << i + 1 << " = " << x[i] << "\n";
        }

        cout << "1) Решить новую матрицу\n";
        cout << "2) Выйти из программы\n";
        cin >> choice;

        while (choice != 1 && choice != 2) {
            cout << "Ошибка! Введите 1 или 2: ";
            cin >> choice;
        }

        if (choice == 2) {
            cout << "\nПрограмма завершена\n";
        }
    } while (choice != 2);

    return 0;
}
```